

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Iulian Pavlov, FAF-241

Report

Laboratory work nr. 5

Files, Directories, and the Storage Stack

Course: Operating Systems

Checked by:
Reiţman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Contents

1 Theoretical Background and Objectives **2**
1.1 Laboratory Purpose and Scope . . . 2
1.2 Inodes, Directory Entries, and Link Mechanics . . . 2
1.3 POSIX Permissions and Directory Semantics . . . 3

2 Part 1: Navigating the Filesystem Hierarchy **4**
2.1 Command Execution and Raw Terminal Session . . . 4
2.2 Answers and Technical Analysis (Part 1) . . . 4

3 Part 2: Inodes, Links, and Storage Allocation **6**
3.1 Command Execution and Raw Terminal Session . . . 6
3.2 Answers and Technical Analysis (Part 2) . . . 7

4 Part 3: Permissions, Umask, and Special Bits **9**
4.1 Command Execution and Raw Terminal Session . . . 9
4.2 Answers and Technical Analysis (Part 3) . . . 9

5 Part 4: Storage Stack, Mounts, and Virtual Filesystems **12**
5.1 Command Execution and Raw Terminal Session . . . 12
5.2 Answers and Technical Analysis (Part 4) . . . 13

6 Part 5: Advanced Investigations (Bonus Tasks) **15**
6.1 Task (a): Magic Numbers and File Format Signatures . . . 15
6.2 Task (b): Auditing SetUID Executables Across the Filesystem . . . 15
6.3 Task (c): Lifecycle of an Unlinked Open File Descriptor . . . 16

7 Conclusions **17**

1 Theoretical Background and Objectives

1.1 Laboratory Purpose and Scope

The primary objective of Laboratory Work Nr. 5 is to investigate the foundational mechanisms of UNIX/Linux storage, the Virtual Filesystem (VFS) layer, inode metadata architecture, and POSIX access control. While high-level file manipulation is abstracted by standard POSIX library calls, operating systems structure persistent and virtual data through layered abstractions: block devices, cryptographic volume mappers, concrete filesystem drivers (e.g., ext4, Btrfs), the VFS abstraction layer, and process-specific file descriptor tables.

This laboratory systematically examines:

- **Filesystem Hierarchy Standard (FHS)**: Organization of root directories, file type identification, the modern unified UsrMerge architecture, and device file interfaces.
- **Inodes and Linking Semantics**: Decoupling directory entries from physical inode records, tracking hard link references (`st_nlink`), resolving broken symbolic links, and examining directory link count propagation.
- **Block Allocation Mechanics**: Granular block allocation units (4 KiB standard) versus sparse file allocation with filesystem holes created via `truncate`.
- **POSIX Discretionary Access Control (DAC)**: Permission mode representation, mathematical umask filtering, identity-based authorization checks, directory traversal permissions (`r` versus `x`), and elevated privilege primitives (SetUID and the Sticky bit).
- **The Linux Storage Hierarchy and VFS**: Mapping NVMe solid-state storage, LUKS encrypted partitions, Btrfs subvolumes, and kernel introspection via the synthetic `procfs` filesystem.
- **Advanced System Diagnostics**: Header-based file format identification via magic numbers, auditing SetUID binaries, and analyzing the kernel lifecycle of unlinked open file descriptors.

1.2 Inodes, Directory Entries, and Link Mechanics

In traditional UNIX filesystems, a file's name and its underlying data are decoupled:

1. **Directory Entry (dentry)**: A simple mapping stored inside a directory associating a human-readable file name with an inode number.
2. **Index Node (inode)**: The internal filesystem data structure storing essential metadata: file type, permissions, owner UID, group GID, file size in bytes, timestamps (`atime`, `mtime`, `ctime`, `btime`), link counter (`st_nlink`), and data block pointers (or B-tree extents).

When a hard link is created with `ln target link`, a new directory entry is written pointing to the existing inode, incrementing `st_nlink`. The `rm` command executes the

POSIX `unlink()` system call, decrementing `st_nlink`. Data blocks and the inode itself are only freed when `st_nlink` drops to zero *and* all active process file descriptors referencing that inode are closed.

Conversely, a symbolic link (soft link) created with `ln -s target link` is an independent file possessing its own inode. Its payload contains solely the text string representing the target file's path. If the target file is removed, the symbolic link becomes dangling.

1.3 POSIX Permissions and Directory Semantics

POSIX DAC evaluates permissions along three identity classes: **User** (owner), **Group**, and **Other**. Each class contains three permission bits:

- **Read (`r`)**: On regular files, permits reading contents. On directories, permits enumerating the list of directory entries (filenames).
- **Write (`w`)**: On regular files, permits modifying contents. On directories, permits creating, renaming, or deleting entries within the directory (provided directory `x` is also present).
- **Execute (`x`)**: On regular files, permits executing the binary or script. On directories, permits *traversing* through the directory to resolve paths and access child inodes.

Special bits modify execution and deletion behavior:

- **SetUID (`s`, octal 4000)**: Executable runs with the effective privileges of the file's owner (typically `root`).
- **Sticky Bit (`t`, octal 1000)**: Applied to world-writable directories (e.g., `/tmp`) to ensure only the file's owner or `root` may delete or rename files.

2 Part 1: Navigating the Filesystem Hierarchy

2.1 Command Execution and Raw Terminal Session

We began by inspecting the root filesystem structure and querying path attributes, file types, and disk consumption across standard system mount points:

```
~/os-lab5 > ls /
Permissions Size User Date Modified Name
lrwxrwxrwx     - root 12 Oct  2025  bin -> usr/bin
drwx------     - root  1 Jan  1970  boot
drwxr-xr-x     - root 12 Oct 14:53  dev
drwxr-xr-x     - root 11 Oct 12:23  etc
drwxr-xr-x     - root 28 Sep 17:23  home
lrwxrwxrwx     - root 12 Oct  2025  lib -> usr/lib
lrwxrwxrwx     - root 12 Oct  2025  lib64 -> usr/lib
drwxr-xr-x     - root 12 Oct  2025  mnt
drwxr-xr-x     - root  1 Oct 22:40  opt
dr-xr-xr-x     - root 12 Oct 14:19  proc
drwxr-x---     - root 28 Sep 17:26  root
drwxr-xr-x     - root 12 Oct 15:08  run
lrwxrwxrwx     - root 12 Oct  2025  sbin -> usr/bin
drwxr-xr-x     - root 28 Sep 17:23  srv
drwxr-xr-x     - root 28 Sep 17:24  swap
dr-xr-xr-x     - root 12 Oct 15:13  sys
drwxrwxrwt     - root 12 Oct 15:14  tmp
drwxr-xr-x     - root 10 Oct 14:40  usr
drwxr-xr-x     - root 11 Oct 12:23  var

~/os-lab5 > ls -ld /etc /etc/passwd/ /bim /dev/null /tmp
"/etc/passwd/": Not a directory (os error 20)
"/bim": No such file or directory (os error 2)
Permissions Size User Date Modified Name
drwxr-xr-x     - root 11 Oct 12:23  /etc
drwxrwxrwt     - root 12 Oct 15:14  /tmp
crw-rw-rw-   1,3 root 12 Oct 14:19  /dev/null

~/os-lab5 > file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically
    linked, interpreter /lib64/ld-linux-x86-64.so.2,
    BuildID[sha1]=4e297d3b427342e1da6b66f5ca0fd279f43f3afe, for GNU/Linux 4.4.0,
    stripped
/etc/passwd: ASCII text
/bin:        symbolic link to usr/bin

~/os-lab5 > du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
376M  /var/log
17G /usr
```

2.2 Answers and Technical Analysis (Part 1)

1. **For each path in the second line, write its type from the first letter of `ls -l`.**
   Answer:

   - `/etc`: First letter is `d`, indicating a **Directory** (`drwxr-xr-x`).

- `/etc/passwd/`: Returned `Not a directory (os error 20)` (`ENOTDIR`). Under POSIX path resolution, appending a trailing slash forces the operating system to resolve the final component as a directory. Because `/etc/passwd` is a regular file (`-`), resolution terminates with error 20.
- `/bim`: Returned `No such file or directory (os error 2)` (`ENOENT`). The path does not exist on disk due to a typographical error for `/bin`.
- `/dev/null`: First letter is `c`, indicating a **Character device file** (`crw-rw-rw-`). Writes to this special node are immediately discarded by the kernel driver, and reads instantly return end-of-file (0 bytes).
- `/tmp`: First letter is `d`, indicating a **Directory** (`drwxrwxrwt`), possessing the Sticky bit (`t`).

2. **What is `/bin` on your system: a directory or a link, and to where?**
   Answer: On this system, `/bin` is a **symbolic link** pointing to `usr/bin` (`bin -> usr/bin`).
   This architectural arrangement implements the modern Linux **UsrMerge** standard. Historically, UNIX separated `/bin` (static rescue utilities needed prior to mounting secondary partitions during boot) from `/usr/bin` (general user software). With modern initramfs loaders mounting the full root filesystem early, separate binary hierarchies are redundant. Consolidating all executable binaries into `/usr/bin` and maintaining `/bin`, `/sbin`, `/lib`, and `/lib64` as symlinks eliminates duplication and simplifies package management.

3. **Find one character device and one block device in `/dev`.**
   Answer:
   - **Character device (`c`)**: `/dev/null` (major 1, minor 3), `/dev/zero`, or `/dev/tty`. Character devices provide unbuffered, sequential byte-stream communication with hardware drivers.
   - **Block device (`b`)**: `/dev/nvme0n1` (major 259, minor 0) or `/dev/dm-0` (major 253, minor 0). Block devices represent random-access persistent storage media operating via fixed-size blocks cached through the kernel page buffer.

4. **Which is bigger on your laptop, `/var/log` or `/usr`, and by roughly how much?**
   Answer:
   The measured directory disk consumption shows:

$$\text{Size of } \texttt{/var/log} = 376 \text{ MB}$$
$$\text{Size of } \texttt{/usr} = 17 \text{ GB} = 17,408 \text{ MB}$$

   Therefore, `/usr` is substantially larger than `/var/log` by approximately **16.6 GB** (roughly **45 times larger**). This differential is expected: `/usr` houses the entire user-space software ecosystem, including shared C libraries, compiler toolchains, graphical desktop assets, and binaries, whereas `/var/log` holds bounded journal logs managed by systemd journal rotation.

3 Part 2: Inodes, Links, and Storage Allocation

3.1 Command Execution and Raw Terminal Session

In this section, we investigated hard and symbolic link mechanics, unlinking behavior, directory reference counts, and sparse file allocation:

```
~/os-lab5 > echo hello > notes.txt
~/os-lab5 > stat notes.txt
  File: notes.txt
  Size: 6           Blocks: 8          IO Block: 4096   regular file
Device: 0,58  Inode: 1100843     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/sunflower)   Gid: ( 1000/sunflower)
Access: 2026-10-12 15:19:45.543385538 +0300
Modify: 2026-10-12 15:19:45.210383395 +0300
Change: 2026-10-12 15:19:45.210383395 +0300
 Birth: 2026-10-12 15:19:45.210383395 +0300

~/os-lab5 > ls -li notes.txt
  inode Permissions Size User      Date Modified Name
1100843 .rw-r--r--     6 sunflower 12 Oct 15:19  notes.txt

~/os-lab5 > ln notes.txt hard.txt
~/os-lab5 > ls -li notes.txt hard.txt
  inode Permissions Size User      Date Modified Name
1100843 .rw-r--r--     6 sunflower 12 Oct 15:19  hard.txt
1100843 .rw-r--r--     6 sunflower 12 Oct 15:19  notes.txt

~/os-lab5 > rm notes.txt
~/os-lab5 > cat hard.txt ; ls -li hard.txt
hello
  inode Permissions Size User      Date Modified Name
1100843 .rw-r--r--     6 sunflower 12 Oct 15:19  hard.txt

~/os-lab5 > ln -s hard.txt soft.txt
~/os-lab5 > ls -li hard.txt soft.txt
  inode Permissions Size User      Date Modified Name
1100843 .rw-r--r--     6 sunflower 12 Oct 15:19  hard.txt
1100898 lrwxrwxrwx     - sunflower 12 Oct 15:22  soft.txt -> hard.txt

~/os-lab5 > cat soft.txt
hello

~/os-lab5 > rm hard.txt
~/os-lab5 > cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
Permissions Size User      Date Modified Name
lrwxrwxrwx     - sunflower 12 Oct 15:22  soft.txt -> hard.txt

~/os-lab5 > mkdir d ; ls -ld d
Permissions Size User      Date Modified Name
drwxr-xr-x     - sunflower 12 Oct 15:23  d

~/os-lab5 > mkdir d/x d/y ; ls -ld d
Permissions Size User      Date Modified Name
drwxr-xr-x     - sunflower 12 Oct 15:23  d

~/os-lab5 > echo x > one.txt ; ls -l one.txt ; du -h one.txt
```

```
Permissions Size User      Date Modified Name
.rw-r--r--     2 sunflower 12 Oct 15:24  one.txt
4.0K  one.txt

~/os-lab5 > truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
Permissions Size User      Date Modified Name
.rw-r--r--  1.1G sunflower 12 Oct 15:24  big.img
0 big.img

~/os-lab5 > rm big.img
```

3.2 Answers and Technical Analysis (Part 2)

1. **Which number in `stat` is the inode, and which tells how many names the file has?**
   Answer:
   - Inode identifier: **`Inode: 1100843`**. This is the unique 64-bit index node key indexing the file's metadata record within filesystem device `0,58`.
   - Name count: **`Links: 1`** (POSIX field `st_nlink`). It represents the number of directory entries that point directly to this inode.

2. **After `rm notes.txt`, why could you still read `hard.txt`? What does `rm` really remove?**
   Answer:
   In Linux, filenames are not the file itself; they are directory entries pointing to an inode. When `ln notes.txt hard.txt` was run, the filesystem created a second directory entry named `hard.txt` referencing the identical inode `1100843`, incrementing `st_nlink` from 1 to 2.
   The `rm` command does not directly delete data blocks; it executes the POSIX `unlink()` system call. `unlink()` deletes the directory entry `notes.txt` and decrements the inode's link count (`st_nlink`) to 1. Because `hard.txt` still points to inode `1100843` and `st_nlink` remains > 0, the kernel preserves the inode and all its physical data blocks on disk. The file contents remained intact and accessible through `hard.txt`. Physical deallocation occurs only when `st_nlink` reaches 0 and no processes hold open descriptors to it.

3. **Why did `cat soft.txt` fail at the end, although `soft.txt` still exists? How big is `soft.txt` and why exactly that size?**
   Answer:
   - **Failure mechanism**: `soft.txt` is a symbolic link possessing its own distinct inode (`1100898`). It stores only the destination string `"hard.txt"`. When `rm hard.txt` unlinked the target, `soft.txt` became a **dangling (broken) symlink**. When `cat soft.txt` is invoked, the kernel attempts path resolution by reading `"hard.txt"` and attempting to open that path. Because `hard.txt` no longer exists, the system call terminates with `ENOENT` (`No such file or directory`).
   - **Nominal file size**: The size of `soft.txt` is exactly **8 bytes**. Under POSIX specification, the size of a symbolic link equals the exact number of bytes in its

stored destination pathname string, excluding any terminating null byte. The string `"hard.txt"` consists of 8 ASCII characters:

$$\texttt{'h', 'a', 'r', 'd', '.', 't', 'x', 't'} \implies 8 \times 1 \text{ byte} = 8 \text{ bytes}.$$

4. **A new directory has 2 links, and 4 after you create two subdirectories. Explain where each link comes from.**
Answer:
When directory `d` is initially created (`mkdir d`):

   (a) **Link 1**: The entry `d` inside the parent directory (`os-lab5/d`) pointing to `d`'s inode.

   (b) **Link 2**: The self-referential entry `.` inside `d` (`d/.`) pointing back to `d`'s inode.

   When subdirectories `d/x` and `d/y` are created (`mkdir d/x d/y`):

   (a) **Link 3**: The parent entry `..` inside `d/x` (`d/x/..`) pointing to `d`.

   (b) **Link 4**: The parent entry `..` inside `d/y` (`d/y/..`) pointing to `d`.

   Consequently, in traditional UNIX filesystems, a directory's link count is always given by:

$$\text{Link Count} = 2 + (\text{Number of Subdirectories}).$$

3. **Why does a 2-byte file use 4.0K, and a 1 GB file use 0?**
Answer:

   - **`one.txt` (2 bytes size, 4.0K disk usage)**: Filesystems allocate storage on physical media in discrete allocation blocks (typically 4096 bytes / 4 KiB). Even though the content of `one.txt` is merely 2 bytes (`'x'` and `'\n'`), the filesystem allocation engine must reserve a full 4 KiB block on disk.
   - **`big.img` (1.1 GB nominal size, 0 disk usage)**: The `truncate -s 1G` command produces a **sparse file**. It adjusts the file length field inside the inode metadata to 10^9 bytes without writing data or allocating physical storage extents. Unwritten ranges are designated by the filesystem driver as "holes". Until real data is written to these offsets, zero physical storage blocks are allocated, so `du -h` reports `0`.

4 Part 3: Permissions, Umask, and Special Bits

4.1 Command Execution and Raw Terminal Session

This section examined POSIX permissions, permission denial mechanics, the mathematical operation of umask, the directory traversal experiment (`box`), and elevated special bits:

```
~/os-lab5 > echo secret > s.txt
~/os-lab5 > chmod 600 s.txt ; ls -l s.txt
Permissions Size User      Date Modified Name
.rw-------     7 sunflower 12 Oct 15:26  s.txt

~/os-lab5 > chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied

~/os-lab5 > chmod u+r s.txt ; cat s.txt
secret

~/os-lab5 > umask ; touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
0022
Permissions Size User      Date Modified Name
drwxr-xr-x     - sunflower 12 Oct 15:27  newdir
.rw-r--r--     0 sunflower 12 Oct 15:27  new.txt

~/os-lab5 > mkdir box ; echo hi > box/f
~/os-lab5 > chmod u=r,go= box ; ls box ; cat box/f
Permissions Size User Date Modified Name
.---------     0 root -             f
cat: box/f: Permission denied

~/os-lab5 > chmod u=x,go= box ; ls box ; cat box/f
Permission denied: box - code: 13
hi

~/os-lab5 > chmod 755 box
~/os-lab5 > ls -l /usr/bin/passwd ; ls -ld /tmp
Permissions Size User Date Modified Name
.rwsr-xr-x   81k root 30 Jul 13:04  /usr/bin/passwd
Permissions Size User Date Modified Name
drwxrwxrwt     - root 12 Oct 15:25  /tmp

~/os-lab5 > cat /etc/shadow
cat: /etc/shadow: Permission denied

~/os-lab5 > ls -l /etc/shadow
Permissions Size User Date Modified Name
.rw-------   947 root 29 Sep 12:13  /etc/shadow
```

4.2 Answers and Technical Analysis (Part 3)

1. **Write 600, 644, 755 and 700 as rwx strings, and give one real use for each.**
   Answer:

Table 1: POSIX Octal Permission Modes and Practical Applications

Octal	Symbolic (`rwx`)	Real-World Operational Use Case
600	`-rw----`	Highly confidential user files: SSH private keys (`~/.ssh/id_ed25519`), private GPG keys, and database credential environment files.
644	`-rw-r-r-`	Publicly accessible non-executable system files: web server assets (`index.html`), system configuration files (`/etc/resolv.conf`), and source code.
755	`-rwxr-xr-x` / `drwxr-xr-x`	Standard system binary executables (`/usr/bin/ls`) and public directories (`/usr/share`, user home directories), enabling general execution and traversal.
700	`-rwx---` / `drwx---`	Confidential private directories: user SSH key storage (`~/.ssh`), GPG keyrings (`~/.gnupg`), or the root user directory (`/root`).

2. **With `chmod 000` you own the file but cannot read it. Why does ownership not help?**
Answer:
POSIX DAC enforces security checks hierarchically during the `open()` system call:

   (a) If the process's effective UID matches the file's owner UID, **only the owner permission bits are evaluated**.

   (b) If the owner bits lack the requested permission (e.g., read `r` is cleared in `000`), the kernel immediately halts evaluation and returns `EACCES` (`Permission denied`). The kernel does not check group or other permissions.

   Ownership grants the authority to *alter* metadata and permissions (via `chmod()`), but does not bypass active permission bits during file I/O operations. To read the file, the owner must explicitly grant themselves read rights via `chmod u+r`.

3. **What is your umask, and how does it explain the permissions of `new.txt` and `newdir`?**
Answer:
The system `umask` is `0022`.
The umask operates as a bitwise mask applied during creation system calls (`open()` with `O_CREAT`, `creat()`, and `mkdir()`):

$$\text{Effective Mode} = \text{Base Mode} \wedge \neg(\text{umask})$$

- **Regular Files**: Base creation mode is `0666` (`rw-rw-rw-`).

$$0666 \wedge \neg(0022) = 0666 \wedge 0755 = 0644 \quad (\texttt{-rw-r-r-}).$$

   This accounts for `new.txt` having permissions `-rw-r-r-`.

- **Directories**: Base creation mode is `0777` (`rwxrwxrwx`).

$$0777 \wedge \neg(0022) = 0777 \wedge 0755 = 0755 \quad (\texttt{drwxr-xr-x}).$$

  This accounts for `newdir` having permissions `drwxr-xr-x`.

The umask strips write access for group and others while preserving owner write access and general read/traversal privileges.

4. **Explain the `box` experiment: what does `r` allow on a directory, and what does `x` allow?**
Answer:

- **Read (`r`) on a directory**: Permits reading the raw directory stream containing the list of filename strings. With `chmod u=r,go= box`, running `ls box` successfully enumerated the file entry `f`. However, without execute (`x`), the kernel cannot traverse the directory to look up the inode behind the name. Thus `cat box/f` failed with `Permission denied`, and `ls -l` displayed unresolvable metadata (`.----- 0 root`).
- **Execute (`x`) on a directory**: Permits directory traversal (searching and path resolution). With `chmod u=x,go= box`, reading the directory index was prohibited (`ls box` yielded `Permission denied`), but because the filename `f` was already known, the kernel successfully traversed through `box`, opened the underlying inode, and `cat box/f` printed `"hi"`.

5. **What do the `s` in `/usr/bin/passwd` and the `t` in `/tmp` mean, and why does `passwd` need the `s`?**
Answer:

- **SetUID bit (`s` in `.rwsr-xr-x`)**: When an executable possessing the SetUID bit is launched, the process executes with the effective UID of the file's owner (`root`) rather than the invoking user.
- **Why `passwd` requires SetUID**: User password hashes are stored inside `/etc/shadow`, which has permissions `600` owned by `root` (`.rw---- root root`). Ordinary users cannot read or write `/etc/shadow`. To allow unprivileged users to update their own credentials, `passwd` runs with elevated root privileges, validates the user's current password, and updates `/etc/shadow` securely before exiting.
- **Sticky Bit (`t` in `drwxrwxrwt`)**: Applied to shared, world-writable directories such as `/tmp`. In standard writable directories, any user can delete or rename any file. The Sticky bit enforces that **only the file owner or `root` may delete or rename a file**, protecting temporary files from unauthorized removal by other users.

5 Part 4: Storage Stack, Mounts, and Virtual Filesystems

5.1 Command Execution and Raw Terminal Session

We examined the system's block device layout, active mount points, filesystem types, and kernel introspection metrics exposed via `procfs`:

```
~/os-lab5 > lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE  MOUNTPOINTS
zram0       252:0    0  15.3G  0 disk  [SWAP]
nvme0n1     259:0    0 476.9G  0 disk
|--nvme0n1p1 259:1    0     2G  0 part  /boot
\--nvme0n1p2 259:2    0 474.9G  0 part
  \--root    253:0    0 474.9G  0 crypt /var/log
                                       /var/cache/pacman/pkg
                                       /home
                                       /

~/os-lab5 > df -h
Filesystem        Size  Used Avail Use% Mounted on
dev               7.5G     0  7.5G   0% /dev
run               7.7G  1.9M  7.7G   1% /run
efivarfs          256K  173K   79K  69% /sys/firmware/efi/efivars
/dev/mapper/root  475G  178G  297G  38% /
tmpfs             7.7G   76M  7.6G   1% /dev/shm
tmpfs             7.7G  6.8M  7.7G   1% /tmp
/dev/mapper/root  475G  178G  297G  38% /home
/dev/mapper/root  475G  178G  297G  38% /var/cache/pacman/pkg
/dev/mapper/root  475G  178G  297G  38% /var/log
/dev/nvme0n1p1    2.0G  527M  1.5G  26% /boot
tmpfs             1.6G  3.6M  1.6G   1% /run/user/1000

~/os-lab5 > mount | grep " / "
/dev/mapper/root on / type btrfs
    (rw,relatime,compress=zstd:3,ssd,space_cache=v2,subvolid=256,subvol=/@)

~/os-lab5 > grep -c processor /proc/cpuinfo
12

~/os-lab5 > head -3 /proc/meminfo
MemTotal:       15993828 kB
MemFree:         3457064 kB
MemAvailable:    8559140 kB

~/os-lab5 > head -5 /proc/self/status
Name: head
Umask:  0022
State:  R (running)
Tgid: 61008
Ngid: 0

~/os-lab5 > ls -l /proc/self/fd
lr-x------     - sunflower 12 Oct 15:33  3 -> /proc/61052/fd
lrwx------     - sunflower 12 Oct 15:33  0 -> /dev/pts/3
lrwx------     - sunflower 12 Oct 15:33  1 -> /dev/pts/3
lrwx------     - sunflower 12 Oct 15:33  2 -> /dev/pts/3
```

5.2 Answers and Technical Analysis (Part 4)

1. **Draw your disk: disk name, partitions, which one is mounted at / and with what file system type.**
   Answer:
   The storage hierarchy of the host laptop is illustrated below:

Physical NVMe SSD: `nvme0n1` (476.9 GB)	
Partition 1: `nvme0n1p1` Size: 2.0 GB Type: EFI System Partition Filesystem: VFAT Mount: `/boot` (26% used)	**Partition 2: `nvme0n1p2`** Size: 474.9 GB Type: LUKS dm-crypt Container Decrypted: `/dev/mapper/root` Filesystem: **Btrfs** (rw, zstd:3) • Subvol `/@` → `/` (Root) • Subvol `/@home` → `/home` • Subvol `/@log` → `/var/log` • Subvol `/@pkg` → `/var/cache/pacman`
RAM Compressed Virtual Block Device: `zram0` (15.3 GB, Swap)	

Figure 1: Host Storage Stack: NVMe Hardware, LUKS Layer, and Btrfs Subvolumes

   - **Disk name**: `nvme0n1` (NVMe Controller 0, Namespace 1).
   - **Partition mounted at /**: Partition `nvme0n1p2`, decrypted via the dm-crypt subsystem as `/dev/mapper/root`.
   - **Filesystem type**: **Btrfs** (mounted with transparent compression `compress=zstd:3`, SSD optimizations, and subvolume `/@`).

2. **How full is your root file system?**
   Answer:
   According to `df -h`, the root filesystem (`/dev/mapper/root`) reports:

   - Total Capacity: **475 GB**
   - Used: **178 GB**
   - Available: **297 GB**
   - Utilization: **38% full**.

3. **How many CPU cores and how much RAM does your laptop have, according to `/proc`?**
   Answer:

   - **CPU Cores**: **12 logical cores / threads** (queried via `grep -c processor /proc/cpuinfo`).
   - **RAM Metrics** (from `/proc/meminfo`):
     - `MemTotal`: $15,993,828$ kB $\approx$ **15.25** GiB ($\sim$ 16 GB physical RAM installed).
     - `MemFree`: $3,457,064$ kB ≈ 3.30 GiB.
     - `MemAvailable`: $8,559,140$ kB $\approx$ **8.16** GiB (memory available for launching new applications without swapping).

4. **`/proc/self/status` showed the name of a program. Which one, and why that one?**
   Answer:
   The program name displayed was **`head`** (`Name:  head`).
   Technical Explanation: `/proc/self` is a dynamic symbolic link implemented by the Linux kernel's `procfs` virtual filesystem driver. Whenever any process accesses `/proc/self`, the kernel evaluates the caller's process credentials and resolves the link to `/proc/<PID>`, where `<PID>` is the process ID of the *currently executing process issuing the read system call*. Because the command executed was `head -5 /proc/self/status`, the active process reading the pseudo-file was the `head` executable itself. Consequently, `/proc/self` resolved to `head`'s PID, returning its process status.

6 Part 5: Advanced Investigations (Bonus Tasks)

The laboratory assignment specifies selecting two advanced tasks for investigation. Below, three in-depth tasks are documented.

6.1 Task (a): Magic Numbers and File Format Signatures

Experimental Procedure and Output:

```
~/os-lab5 > gzip -c /etc/passwd > notes.txt
~/os-lab5 > file notes.txt
notes.txt: gzip compressed data, was "passwd", last modified: Tue Sep 29 09:13:07
    2026, from Unix, original size modulo 2^32 1726
~/os-lab5 > head -c 16 notes.txt | od -t x1 -A n
 1f 8b 08 08 23 81 bb 6a 00 03 70 61 73 73 77 64
```

Analysis:
UNIX systems do not rely on filename extensions to determine file types; rather, the `file` utility uses `libmagic` to inspect the leading byte sequence against a database of known file headers.

- The first two bytes, **`0x1F 0x8B`** (`1f 8b` in hexadecimal, or octal `037 0213`), constitute the standardized **gzip magic number header** defined in RFC 1952.
- The third byte (`0x08`) denotes the DEFLATE compression method.
- The fourth byte (`0x08`) indicates that the `FNAME` flag is set, enabling `file` to extract the original compressed filename (`"passwd"`).

6.2 Task (b): Auditing SetUID Executables Across the Filesystem

Experimental Procedure and Output:

```
~/os-lab5 > find / -xdev -perm -4000 -type f 2>/dev/null
/usr/bin/ksu
/usr/bin/unix_chkpwd
/usr/bin/chage
/usr/bin/gpasswd
/usr/bin/passwd
/usr/bin/chfn
/usr/bin/chsh
/usr/bin/mount
/usr/bin/newgrp
/usr/bin/su
/usr/bin/umount
/usr/bin/sudo
/usr/bin/pkexec
/usr/bin/fusermount3
/usr/bin/crontab
/usr/lib/dbus-daemon-launch-helper
/usr/lib/Xorg.wrap
/usr/lib/ssh/ssh-keysign
/usr/lib/chromium/chrome-sandbox
/usr/lib/electron43/chrome-sandbox
/opt/brave-bin/chrome-sandbox
```

Analysis:
The audit identified standard administrative and authentication binaries (`passwd`, `sudo`, `su`, `mount`, `pkexec`, `chsh`) alongside sandboxing helpers (`/opt/brave-bin/chrome-sandbox`, `/usr/lib/chromium/chrome-sandbox`).

- Web browsers utilize dedicated SetUID root helper executables to construct secure, isolated sandbox environments.
- The sandbox helper initializes Linux unprivileged user namespaces (`CLONE_NEWUSER`), PID namespaces, and seccomp system call filters before irrevocably dropping all root capabilities via `setuid()` and transferring execution to the browser renderer.

6.3 Task (c): Lifecycle of an Unlinked Open File Descriptor

Experimental Procedure and Output:

```
~/os-lab5 > exec 3< notes.txt
~/os-lab5 > rm notes.txt
~/os-lab5 > ls -l /proc/$$/fd
lr-x------ 1 sunflower sunflower 64 Oct 12 15:45 3 ->
    /home/sunflower/os-lab5/notes.txt (deleted)
~/os-lab5 > exec 3<&-
```

Analysis:
When `rm notes.txt` is invoked while file descriptor 3 remains open, the directory entry is unlinked, reducing `st_nlink` to 0. Inspecting `/proc/$$/fd/3` displays the path annotated with `(deleted)`. Under POSIX semantics, the operating system preserves the underlying inode and data blocks in storage memory as long as at least one active file descriptor references it. The storage space is truly deallocated and marked available for reallocation only when descriptor 3 is closed with `exec 3<&-` or the owning process terminates.

7 Conclusions

Laboratory Work Nr. 5 provided rigorous, practical experience with the internal mechanics of the UNIX/Linux storage hierarchy, inode structures, and the POSIX security architecture:

1. **Separation of Names and Metadata**: Linux strictly decouples human-readable filenames from internal inode records. Hard links provide multiple directory entry names referencing a shared inode, whereas symbolic links are independent pointer files whose byte size reflects the length of their destination pathname.

2. **System Call Semantics of Unlinking**: The `rm` command does not directly delete data blocks; it executes `unlink()`, decrementing the inode's reference count. Physical storage blocks are reclaimed only when the link count reaches zero and all open file descriptors to the inode are closed.

3. **Granular vs. Sparse Allocation**: Storage systems manage allocations in minimum block units (4 KiB), creating internal slack for tiny files, while sparse files exploit hole metadata to allocate gigabyte-scale address ranges with zero physical block consumption.

4. **Precise POSIX Access Control**: Discretionary Access Control evaluates permissions strictly according to identity tiers. Ownership allows metadata modification but does not bypass active permission bits. Directory permissions distinguish between entry enumeration (`r`) and path traversal/inode lookup (`x`).

5. **Layered Storage Architecture and Kernel Virtualization**: Modern Linux architectures combine NVMe block devices, dm-crypt LUKS encryption, and advanced Btrfs subvolumes, while the `procfs` synthetic filesystem demonstrates how the operating system exposes kernel state, hardware topology, and process descriptors through the universal UNIX file API.